

Creación de una evaluación académica: Programación Orientada a Objetos en C++



Integrantes:
Joaquín Fuenzalida C.
Benjamín Vallejos V.

Profesor:
Juan Calderón M.

Introducción

Este informe presenta la realización de un programa en C++ donde se le permitirá al usuario poder crear y gestionar una evaluación con preguntas de tipo verdadero/falso y de opción múltiple, donde se verán encasilladas por la taxonomía de Bloom.

La motivación detrás de la elaboración de este trabajo consiste en resolver de una manera tecnológica y sencilla la creación de una prueba, donde se le permitirá a nuestro cliente la creación, modificación y visualización de esta misma. Por otra parte, el aprendizaje y comprensión de un nuevo lenguaje de programación es otra de las motivaciones detrás de este proyecto, entendiendo de mejor manera los nuevos conceptos de clases, herencia y punteros, por ejemplo.

Este informe está estructurado de la siguiente forma: Primero se explicará la elección de clases y atributos que se usaron dentro de nuestro sistema. Después, se explicará el funcionamiento de nuestro sistema, comentando partes importantes del código y funcionalidades nuevas que se comprenden dentro de este. Y finalmente se presentarán las conclusiones generales del proyecto.

Objetivo General

-Crear y desarrollar un programa mediante el lenguaje de C++ y la programación orientada a objetos, en donde se elabore una prueba o examen consistiendo de preguntas de alternativas y preguntas de verdadero o falso, siendo estas clasificadas mediante la taxonomía de Bloom

Objetivos Específicos:

- Implementar el uso de clases para la representación de las preguntas de distinto tipo (Verdadero/Falso y Alternativas).
- Realización de un menú interactivo para la fácil manipulación del usuario.
- Implementación de funcionalidades de visualización, modificación y eliminación de las preguntas creadas por el usuario, además del cálculo del tiempo que podría demorar el alumno para responder.

Descripción de la solución

Dentro del desarrollo del programa solicitado, se tomaron en cuenta la creación de las siguientes clases esenciales:

1. **Pregunta:** Representa la clase “padre”, la cual heredará los atributos privados (encapsulamiento) “enunciado”, “tiempo estimado” y “taxonomía”. Clase fundamental que maneja y representa la estructura de una pregunta de prueba.
2. **PreguntaVoF:** Clase “hija”, la que hereda atributos de clase Pregunta y a su vez tiene atributos propios y privados tales como “respuesta correcta” y “justificación”, atributos necesarios que caracterizan a una pregunta de verdadero/falso.
3. **PreguntaAlt:** Clase “hija”, la que hereda atributos de clase Pregunta, teniendo atributos propios y privados tales como “opciones” y “respuesta correcta”.
4. **Prueba:** Representa la estructura clave de una evaluación, la que contiene los tipos de preguntas tales como verdadero/falso y alternativas (Clases que forman parte de la clase Prueba) con el uso de métodos utilizados para su correcta implementación y funcionamiento.
5. **Menu:** Creación de clase menú, la que contiene de manera organizada las secciones a escoger por el usuario, tales como: agregar preguntas, mostrar preguntas, buscar por nivel taxonómico, cálculo de tiempo total de evaluación, borrar ítem, actualizar ítem y salir del programa.

Detalles del modelo:

- **Clase Pregunta:** Se crea utilizando la aplicación del encapsulamiento en sus atributos (private), las cuales pueden ser manipuladas dentro del public con la utilización de métodos tales como Getters y Setters.

Atributos:

- enunciado: texto que contiene el planteo de una pregunta (string, private).
- tiempo estimado: variable que almacena el tiempo estimado total de cada pregunta (int, private).
- taxonomia: almacena el nivel taxonómico que tendrán las preguntas en cuestión (string, private)

Métodos:

- Uso de getters y setters para cada atributo
- mostrarPregunta(): Muestra la información de la pregunta en consola. (void, public)

- **Clase PreguntaVoF:** Contiene los atributos y métodos para la construcción de una pregunta Verdadero y Falso. Esta clase hereda atributos de la clase “padre” Pregunta.

Atributos propios:

- respuestaCorrecta: Retorna “true” siendo la respuesta correcta, “false” en el caso contrario (bool, private)
- justificacion: Entrega la justificación de la respuesta a la pregunta (string, private)

Métodos:

- Getters y setters para cada atributo (public)
- mostrarPregunta: Entrega en consola la pregunta ingresada (void, public)

- **Clase PreguntaAlt:** Contiene los atributos y métodos para la construcción de una pregunta de selección múltiple. Esta clase hereda atributos de la clase “padre” Pregunta.

Atributos propios:

- opciones: Contiene las alternativas a escoger (de tipo vector<string>, private)
- respuestaCorrecta: Almacena la respuesta correcta de la pregunta (int, private)

Métodos:

- Uso de getters y setters para cada atributo
- mostrarPregunta: Muestra la pregunta estructurada en consola (void, public)
- **Clase Prueba:** Contiene las clases “Pregunta”, “PreguntaAlt” y “PreguntaVoF”, la cual es la base del programa y contiene la estructura de este.

Atributos propios:

- preguntasVoF (vector<PreguntaVoF*>, p)
- preguntasAlt (vector<PreguntaAlt*>, p)

Métodos:

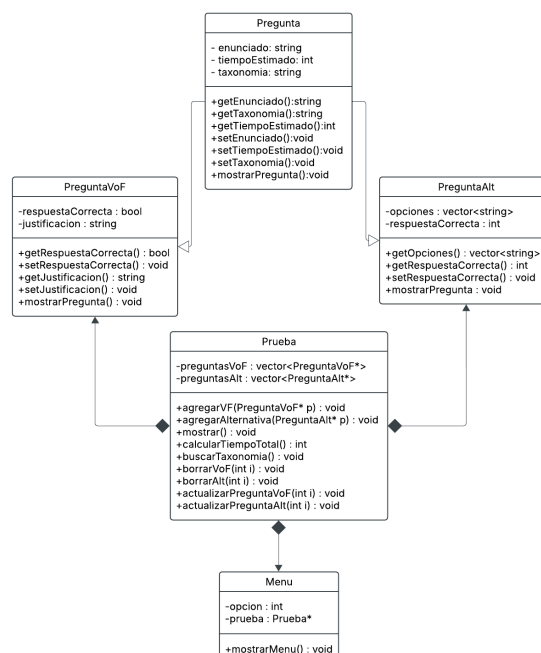
- agregarVF(PreguntaVoF* p): Crea una pregunta Verdadero y Falso (void, public)
- agregarAlternativa(PreguntaAlt* p): Crea una pregunta de alternativas (void, public)
- mostrar(): Muestra las preguntas creadas (void, public)
- calcularTiempoTotal(): Calcula el tiempo según el tipo de pregunta (int, public)
- buscarTaxonomia(): Busca preguntas según su taxonomía (void, public)
- borrarVoF(int i): Borra la pregunta de Verdadero/Falso (void, public)
- borrarAlt(int i): Borra la pregunta de alternativas (void, public)
- actualizarPreguntaVoF(int i): Actualiza la pregunta verdadero/falso (void, public)
- **Clase Menu:** Clase que contiene las secciones solicitadas, tales como agregar preguntas verdadero y falso + alternativas, mostrar todas las preguntas, buscar por nivel taxonómico, cálculo de tiempo total de la evaluación, borrar ítem, actualizar ítem y salir del programa.

Atributos:

- opcion: Muestra las secciones de un menú a escoger por el usuario (int, private)
- prueba: Llama a la dirección de memoria prueba (Prueba*)

Métodos:

- mostrarMenu(): Método que muestra todas las secciones del menú y ejecutando el programa (void, public)



Conclusión

La realización de este trabajo permitió aplicar los conocimientos aprendidos a lo largo de las clases, como también el aprendizaje adquirido fuera de estas con el uso de las clases, herencias, atributos (privados y públicos) y sus métodos.

Ya finalizado el código, podemos decir que se han cumplido todos, implementando las clases necesarias y funcionamiento general para la creación de pruebas, tales como las clases de prueba y menú junto a sus requeridos métodos o las preguntas y sus atributos característicos.

Este trabajo nos ayudó a comprender de mejor manera la estructuración de un programa en C++ relacionado a la programación orientada a objetos. A su vez, conceptos o funcionalidades nuevas que aprendimos dentro del camino para el completo funcionamiento del código.